

2 - PHP com Laravel

Objetivo:

Este curso tem como objetivo ensinar os alunos a instalar, configurar e preparar o ambiente para desenvolvimento com o framework Laravel, cobrindo os conceitos básicos necessários para iniciar o desenvolvimento de aplicações web.

Conteúdo Programático:

Introdução ao Laravel e Preparação do Ambiente

Introdução ao Laravel

O que é o Laravel?

Laravel é um framework PHP para desenvolvimento de aplicações web que segue o padrão MVC (Model-View-Controller). Ele foi projetado para facilitar o processo de desenvolvimento, oferecendo uma sintaxe elegante e ferramentas robustas que ajudam os desenvolvedores a escrever código limpo e eficiente. Laravel é conhecido por sua facilidade de uso, recursos avançados e uma grande comunidade de desenvolvedores.

Principais Características:

- MVC: Arquitetura que separa a lógica de negócios, a apresentação e os dados da aplicação.
- Eloquent ORM: Um mapeador objeto-relacional (ORM) que facilita a interação com bancos de dados.
- Artisan CLI: Uma interface de linha de comando que automatiza tarefas comuns de desenvolvimento.
- Blade Template Engine: Um mecanismo de template simples e poderoso para criar views.
- Middleware: Permite a filtragem de requisições HTTP.
-

História e Evolução do Laravel

Laravel foi criado por Taylor Otwell e lançado pela primeira vez em junho de 2011. O objetivo inicial era fornecer uma alternativa mais avançada e simples ao framework CodeIgniter, que era bastante popular na época.

Evolução do Laravel:

- Laravel 1: Lançado em 2011, introduziu recursos básicos como roteamento, views e sessões.
- Laravel 2: Introduzido em 2011, adicionou suporte a controllers e o ORM Eloquent.
- Laravel 3: Lançado em 2012, trouxe o Artisan CLI, suporte a migrações e pacotes.
- Laravel 4: Lançado em 2013, foi uma reescrita completa, introduzindo o Composer para gerenciamento de dependências.
- Laravel 5: Lançado em 2015, trouxe mudanças significativas como middleware, contract interfaces e Elixir para gerenciamento de ativos.
- Laravel 6, 7 e 8: Lançados entre 2019 e 2020, introduziram novos recursos como Laravel Vapor (deploy serverless), melhor integração com front-end e melhorias na usabilidade e performance.
- Laravel 9 e além: Continuam a aprimorar a usabilidade, performance e introduzir novas funcionalidades de acordo com as necessidades modernas de desenvolvimento web.

Vantagens do Uso do Laravel

Laravel oferece diversas vantagens que o tornam uma escolha popular entre desenvolvedores web:

1. Sintaxe Elegante:
 - Laravel é conhecido por sua sintaxe limpa e expressiva, que facilita a escrita e manutenção do código.
2. Ecossistema Rico:
 - Possui uma vasta gama de pacotes e bibliotecas que simplificam o desenvolvimento de funcionalidades comuns, como autenticação, filas de tarefas, eventos e muito mais.
3. Eloquent ORM:
 - O Eloquent ORM torna a interação com bancos de dados intuitiva e eficiente, permitindo aos desenvolvedores escrever consultas em PHP em vez de SQL.
4. Artisan CLI:

- A ferramenta de linha de comando Artisan facilita a execução de tarefas repetitivas, como geração de código boilerplate, migrações de banco de dados e testes.
 - 5. Blade Template Engine:
 - O Blade é um mecanismo de template poderoso e simples que permite a criação de views dinâmicas de maneira eficiente.
 - 6. Segurança:
 - Laravel cuida de muitos aspectos de segurança, incluindo proteção contra SQL Injection, Cross-Site Scripting (XSS), e Cross-Site Request Forgery (CSRF).
 - 7. Comunidade e Suporte:
 - Laravel possui uma grande comunidade de desenvolvedores e uma excelente documentação, facilitando a resolução de problemas e a aprendizagem contínua.
 - 8. Flexibilidade:
 - Laravel é altamente configurável e extensível, permitindo que os desenvolvedores ajustem e estendam o framework de acordo com as necessidades específicas de seus projetos.
 - 9. Performance:
 - O framework é otimizado para desempenho e pode ser facilmente integrado com tecnologias de cache e otimização.
 - 10. Atualizações Regulares:
 - Laravel é mantido ativamente com lançamentos regulares que trazem novas funcionalidades, melhorias de desempenho e correções de bugs.
-

Preparação do Ambiente de Desenvolvimento

Para desenvolver com Laravel, você precisa configurar o ambiente de desenvolvimento com os seguintes requisitos:

1. PHP 8.x
2. Composer
3. Servidor Web (Apache ou Nginx)
4. Banco de Dados (MySQL ou PostgreSQL)

PHP 8.x

Laravel requer PHP 8.x. Abaixo estão os passos para instalar o PHP no Windows, macOS e Linux.

Windows:

1. Baixe o PHP de [php.net](https://www.php.net).
2. Extraia o arquivo ZIP em uma pasta, por exemplo, C:\src\php.
3. Adicione a pasta PHP ao PATH do sistema:
 - Clique com o botão direito em "Este Computador" e selecione "Propriedades".
 - Clique em "Configurações avançadas do sistema".
 - Na aba "Avançado", clique em "Variáveis de Ambiente".
 - Em "Variáveis do sistema", encontre a variável Path, selecione-a e clique em "Editar".
 - Adicione C:\src\php ao final da lista de valores.

Verifique a instalação abrindo o Prompt de Comando e digitando:

```
php -v
```

Composer

Composer é um gerenciador de dependências para PHP. Ele permite declarar as bibliotecas das quais seu projeto depende e gerencia (instala/atualiza) elas para você.

Instalação do Composer

Windows:

1. Baixe o instalador do Composer de getcomposer.org.
2. Execute o instalador e siga as instruções na tela.

Verifique a instalação abrindo o Prompt de Comando e digitando:

```
composer -v
```

PostgreSQL:

Windows:

1. Baixe o instalador do PostgreSQL de [postgresql.org](https://www.postgresql.org).
2. Execute o instalador e siga as instruções na tela.

Instalação do Laravel via Composer

Para instalar o Laravel, navegue até o diretório onde você deseja criar o seu novo projeto Laravel. Use o comando `cd` para mudar de diretório.

Em seguida, execute o seguinte comando para instalar o Laravel:

```
composer create-project --prefer-dist laravel/laravel meu-primeiro-projeto-laravel
```

Navegar até o Diretório do Projeto

Depois que o Laravel for instalado, navegue até o diretório do seu projeto:

```
cd meu-primeiro-projeto-laravel
```

Estrutura de Diretórios do Laravel

Após a instalação, a estrutura de diretórios do Laravel será semelhante a esta:

```
meu-primeiro-projeto-laravel/
```

```
|— app/
|   |— Console/
|   |— Exceptions/
|   |— Http/
|   |— Models/
|   |— Providers/
|— bootstrap/
|   |— cache/
|— config/
|— database/
|   |— factories/
|   |— migrations/
|   |— seeders/
|— public/
|— resources/
|   |— css/
|   |— js/
|   |— views/
|— routes/
|— storage/
|   |— app/
|   |— framework/
```

```
| |— logs/
|— tests/
|— vendor/
|— .env
|— .gitignore
|— artisan
|— composer.json
|— package.json
|— phpunit.xml
|— README.md
```

Configuração Inicial do Projeto Laravel com PostgreSQL

Configurar o Arquivo .env

O arquivo .env é usado para configurar variáveis de ambiente do seu projeto. Abra o arquivo .env no editor de texto de sua escolha (por exemplo, Visual Studio Code).

Ajuste as configurações de banco de dados para usar o PostgreSQL:

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=postgres
DB_USERNAME=postgres
DB_PASSWORD=postgres
```

Ajustar o banco de Dados da Aplicação usando o artisan migrate

```
php artisan migrate
```

Gerar a Chave da Aplicação

Laravel usa uma chave de aplicação para várias funcionalidades internas. Para gerar uma chave de aplicação, execute o seguinte comando:

```
php artisan key:generate
```

Este comando atualizará automaticamente o valor APP_KEY no arquivo .env.

Criar um "Hello World" em Laravel

Criar uma Rota

Abra o arquivo routes/web.php e adicione uma rota para exibir "Hello World":

```
<?php
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return 'Hello World';
});
```

Iniciar o Servidor de Desenvolvimento

Para iniciar o servidor de desenvolvimento embutido do Laravel, execute:

```
php artisan serve
```

Você verá uma saída indicando que o servidor está rodando:

```
Starting Laravel development server: http://127.0.0.1:8000
```

Abra um navegador web e visite <http://127.0.0.1:8000> para ver a mensagem "Hello World".

Conceitos Básicos do Laravel

Introdução ao Artisan CLI

O que é Artisan?

Artisan é a interface de linha de comando (CLI) incluída com o Laravel. Ele fornece um conjunto de comandos úteis para ajudar no desenvolvimento da sua aplicação. Estes comandos podem ser usados para realizar várias tarefas, como a criação de arquivos de migração, a execução de testes, a criação de controladores e a depuração de problemas. Artisan é construído em cima do Symfony Console, um componente robusto e flexível para a criação de aplicações de linha de comando.

Comandos Básicos do Artisan

Listar Comandos Disponíveis:

```
php artisan list
```

Este comando lista todos os comandos Artisan disponíveis.

Criar Arquivos (make) com Artisan CLI

O Artisan CLI do Laravel fornece uma série de comandos make que ajudam a gerar automaticamente a estrutura de arquivos necessários para diversas partes da sua aplicação. Esses comandos economizam tempo e garantem que você siga as convenções do Laravel.

Comandos make Comuns

- Criar um Controlador : Controladores são usados para agrupar a lógica relacionada a uma rota específica ou um grupo de rotas.

php artisan make:controller UserController

Isso cria um arquivo UserController.php em app/Http/Controllers.

- Criar um Modelo: Modelos são usados para interagir com as tabelas do banco de dados.

php artisan make:model User

Isso cria um arquivo User.php em app/Models.

- Criar uma Migração: Migrações são usadas para criar e modificar tabelas no banco de dados.

php artisan make:migration create_users_table

Isso cria um arquivo de migração em database/migrations.

- Criar um Seeder: Seeders são usados para popular o banco de dados com dados de teste ou padrão.

php artisan make:seeder UserSeeder

Isso cria um arquivo UserSeeder.php em database/seeders.

- Criar uma Factory: Factories são usadas para criar modelos de dados de teste de forma automatizada.

php artisan make:factory UserFactory

Isso cria um arquivo UserFactory.php em database/factories.

- Criar um Middleware: Middlewares são usados para filtrar requisições HTTP antes que elas alcancem o seu controlador.

php artisan make:middleware VerClima

Isso cria um arquivo VerClima.php em app/Http/Middleware.

Executar Migrações (migrate)

As migrações no Laravel fornecem uma maneira conveniente de criar e gerenciar o esquema do banco de dados. Com elas, você pode definir as tabelas e colunas de forma programática e versionada, facilitando a manutenção e evolução do banco de dados durante o ciclo de vida do seu aplicativo.

Comandos Básicos de Migração:

- *php artisan migrate*

O comando migrate aplica todas as migrações pendentes ao banco de dados. Este comando lê todas as migrações na pasta database/migrations que ainda não foram executadas e as aplica ao banco de dados. As migrações são executadas em ordem cronológica com base no timestamp do nome do arquivo de migração.

- *php artisan migrate:rollback*

O comando migrate:rollback reverte a última operação de migração, desfazendo as alterações aplicadas na última execução do migrate. Isso desfaz as últimas "lote" de migrações aplicadas. Um lote é um grupo de migrações que foram executadas juntas.

- *php artisan migrate:refresh*

O comando migrate:refresh primeiro reverte todas as migrações e depois as executa novamente. Este comando é útil quando você deseja redefinir seu banco de dados para um estado inicial, por exemplo, durante o desenvolvimento.

Comandos Adicionais de Migração

- *php artisan migrate:reset*

O comando migrate:reset reverte todas as migrações, removendo todas as tabelas criadas.

- *php artisan migrate:status*

O comando migrate:status exibe o status de cada migração, mostrando se foi aplicada ou não.

Roteamento Laravel

O roteamento no Laravel é usado para definir como o aplicativo deve responder a várias solicitações HTTP. Ele permite mapear URLs específicas para ações específicas em controladores ou funções de retorno de chamada.

Definição de Rotas Básicas

As rotas são definidas no arquivo `routes/web.php` para rotas web e `routes/api.php` para rotas de API. Vamos focar em *routes/web.php*:

Rotas Simples

```
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});
```

Neste exemplo, a rota GET / retorna a view welcome.

Métodos de Rota (GET, POST, PUT, DELETE)

O Laravel suporta todos os métodos HTTP comumente usados.

```
Route::get('/usuarios', 'UserController@index');
Route::post('/usuarios', 'UserController@store');
Route::put('/usuarios/{id}', 'UserController@update');
Route::delete('/usuarios/{id}', 'UserController@destroy');
```

Parâmetros de Rota

Rotas podem aceitar parâmetros que são passados para a ação do controlador.

Parâmetros Obrigatórios

```
Route::get('/usuarios/{id}', function ($id) {
    return "User ID: " . $id;
});
```

Parâmetros Opcionais

```
Route::get('/usuarios/{nome?}', function ($nome = 'Visitante') {
    return "Olá, " . $nome;
```

```
});
```

Views e Blade Template Engine

O que é Blade?

Blade é o motor de templates do Laravel, projetado para ser simples e poderoso. Ele permite a utilização de uma sintaxe limpa e fácil de entender para definir layouts e incluir lógica condicional e loops dentro das suas views.

Sintaxe básica do Blade

Blade utiliza tags especiais para incorporar PHP nas suas views, o que torna a sintaxe mais legível e evita conflitos com código HTML.

- Exibição de Dados: Utiliza `{{ }}` para exibir dados.

```
<h1>{{ $title }}</h1>
<p>{{ $description }}</p>
```

- Lógica Condicional: Utiliza `@if`, `@elseif`, `@else`, e `@endif`.

```
@if ($user->isAdmin())
    <p>Welcome, admin!</p>
@else
    <p>Welcome, user!</p>
@endif
```

- Laços de Repetição: Utiliza `@foreach`, `@for`, `@while`, e `@endforeach`.

```
@foreach ($users as $user)
    <p>{{ $user->name }}</p>
@endforeach
```

- Comentários: Utiliza `{{-- --}}` para adicionar comentários que não serão exibidos no HTML renderizado.

```
{{-- This is a Blade comment --}}
```

Incluindo templates e componentes Blade

- Incluindo templates: incluir outras views dentro de uma view usando a diretiva `@include`.

```

@include('header')
<h1>{{ $title }}</h1>
<p>{{ $description }}</p>
@include('footer')

```

Layouts e seções: Blade facilita a criação de layouts reutilizáveis. Você pode definir um layout base e estender esse layout em suas views.

- Definindo um layout base (layouts/app.blade.php):

```

<!DOCTYPE html>
<html>
<head>
    <title>@yield('title')</title>
</head>
<body>
    @include('partials.nav')
    <div class="container">
        @yield('content')
    </div>
    @include('partials.footer')
</body>
</html>

```

- Estendendo um layout (pages/home.blade.php):

```

@extends('layouts.app')
@section('title', 'Home Page')
@section('content')
    <h1>Welcome to the Home Page</h1>
    <p>This is the content of the home page.</p>
@endsection

```

Componentes Blade

Componentes permitem criar blocos de código reutilizáveis, facilitando a manutenção e reutilização de partes do layout.

- Definindo um componente (resources/views/components/alert.blade.php):

```
<div class="alert alert-{{ $type }}">
    {{ $slot }}
</div>
```

- Usando um componente em uma view:

```
<x-alert type="success">
    Operation completed successfully!
</x-alert>
```

- Incluindo views com dados: pode passar dados para as views incluídas usando a diretiva @include.

```
@include('partials.header', ['title' => 'Page Title'])
```

- No arquivo de inclusão (partials/header.blade.php):

```
<h1>{{ $title }}</h1>
```

Exemplos Práticos

Exemplo 1: Layout com Cabeçalho e Rodapé

Exemplo 2: Usando Componentes

Modelos e Eloquent ORM

Criação de Modelos

Os modelos no Laravel representam as tabelas no banco de dados e fornecem uma interface para interagir com os dados. Através dos modelos, você pode consultar e manipular dados de forma eficiente e intuitiva.

Para Criar um Modelo

Para criar um modelo, devemos usar o comando Artisan make:model.

```
php artisan make:model "NomeModel"
```

Esse comando cria um arquivo de modelo em app/Models/NomeModel.php.

Devemos Editar o arquivo NomeMode.php para definir os atributos que podem ser atribuídos em massa.

app/Models/NomeMode.php

O que é Eloquent?

Eloquent é o ORM (Object-Relational Mapper) do Laravel, que proporciona uma forma simples e elegante de interagir com o banco de dados. Ele permite definir modelos e relacionamentos usando uma sintaxe expressiva e fluente, facilitando a manipulação de dados

- Criação de Modelos

```
php artisan make:model User
```

Isso criará um arquivo de modelo em app/Models/User.php.

Adicionando uma Tabela e Relacionamentos:

- Migration (criação da tabela):

```
php artisan make:migration create_users_table
```

- Edite a migration gerada para definir a estrutura da tabela:

```
use Illuminate\Database\Migrations\Migration;  
use Illuminate\Database\Schema\Blueprint;  
use Illuminate\Support\Facades\Schema;
```

```
class CreateUsersTable extends Migration  
{  
    public function up()  
    {  
        Schema::create('users', function (Blueprint $table) {  
            $table->id();  
            $table->string('name');  
            $table->string('email')->unique();  
            $table->string('password');  
            $table->timestamps();  
        });  
    }  
}
```

```

    });
}

    public function down()
    {
        Schema::dropIfExists('users');
    }
}

```

Controladores de Recursos

Controladores de recursos permitem definir rotas para operações CRUD de forma simplificada em um único comando. Eles ajudam a manter seu código organizado e facilitam a implementação de funcionalidades comuns.

```
Route::resource('usuarios', 'UserController');
```

Isso cria todas as rotas necessárias para operações CRUD:

```

GET /usuarios
GET /usuarios/create
POST /usuarios
GET /usuarios/{id}
GET /usuarios/{id}/edit
PUT/PATCH /usuarios/{id}
DELETE /usuarios/{id}

```

Comando:

```
php artisan make:controller UserController --resource
```

Isso gera um controlador com métodos padrão para as operações CRUD.

O controlador gerado incluirá os seguintes métodos:

- index: Exibir uma lista de recursos
- create: Mostrar o formulário para criar um novo recurso
- store: Salvar um novo recurso
- show: Exibir um recurso específico
- edit: Mostrar o formulário para editar um recurso específico

- update: Atualizar um recurso específico
- destroy: Remover um recurso específico

Exemplo de Aplicação: Criação de um Model, um Migrate e um Controller para Produto

Criação do Modelo e Migração

Utilize o Artisan para criar um modelo chamado Produto com uma migração associada:

```
php artisan make:model Produto -m
```

Isso criará dois arquivos:

- O modelo em app/Models/Produto.php
- A migração em database/migrations/xxxx_xx_xx_create_produtos_table.php

Definição da Migração

Edite o arquivo de migração criado em database/migrations

Código:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

class CreateProdutosTable extends Migration
{
    /**
     * Run the migrations.
     */
}
```



```

    * @return void
    */
    public function up()
    {
        Schema::create('produtos', function (Blueprint $table) {
            $table->id();
            $table->string('nome');
            $table->text('descricao');
            $table->decimal('preco', 8, 2);
            $table->integer('quantidade');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     *
     * @return void
     */
    public function down()
    {
        Schema::dropIfExists('produtos');
    }
}

```

Exemplo para ProdrutoController:

```
php artisan make:controller ProdutoController --resource
```

Código Gerado:

```

<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class ProdutoController extends Controller

```

```
{  
  
    /**  
     * Display a listing of the resource.  
     */  
    public function index()  
    {  
        //  
    }  
  
    /**  
     * Show the form for creating a new resource.  
     */  
    public function create()  
    {  
        //  
    }  
  
    /**  
     * Store a newly created resource in storage.  
     */  
    public function store(Request $request)  
    {  
        //  
    }  
  
    /**  
     * Display the specified resource.  
     */  
    public function show(string $id)  
    {  
        //  
    }  
  
    /**  
     * Show the form for editing the specified resource.  
     */  
    public function edit(string $id)  
    {  
        //  
    }  
}
```

```

    /**
     * Update the specified resource in storage.
     */
    public function update(Request $request, string $id)
    {
        //
    }

    /**
     * Remove the specified resource from storage.
     */
    public function destroy(string $id)
    {
        //
    }
}

```

Usando Seeders

Seeders são usados para popular o banco de dados com dados de teste ou iniciais. No Laravel, podemos criar seeders e usar a biblioteca Faker para gerar grandes volumes de dados aleatórios para testes.

- Criar um Seeder para o Modelo Produto

php artisan make:seeder ProdutoSeeder

Esse Comando irá criar o arquivo *database/seeders/ProdutoSeeder.php*

- Edite o arquivo ProdutoSeeder.php para gerar 50 produtos com valores aleatórios e nomes sequenciais.

database/seeders/ProdutoSeeder.php

```
namespace Database\Seeders;
```

```

use Illuminate\Database\Seeder;
use App\Models\Produto;
use Faker\Factory as Faker;

class ProdutoSeeder extends Seeder
{
    public function run()
    {
        $faker = Faker::create();

        // Gerar 50 produtos
        for ($i = 1; $i <= 50; $i++) {
            Produto::create([
                'nome' => 'Produto ' . $i,
                'descricao' => $faker->sentence,
                'preco' => $faker->randomFloat(2, 1, 1000),
                'quantidade' => $faker->numberBetween(1, 100),
            ]);
        }
    }
}

```

obs:

- Faker: Utilizamos o Faker para gerar dados aleatórios. O Faker é uma biblioteca que permite a geração de dados fictícios.
 - Laço de Repetição: Um laço for é usado para gerar 50 produtos. A cada iteração, um novo produto é criado com valores aleatórios para descricao, preco, e quantidade, enquanto o nome é sequencial (Produto 1, Produto 2, etc.).
- Registrar o Seeder no DatabaseSeeder: Edite o arquivo DatabaseSeeder.php para incluir o seeder ProdutoSeeder.

database/seeder/DatabaseSeeder.php

```

public function run()

```

```
{  
    $this->call([  
        ProdutoSeeder::class,  
    ] );  
}
```

- Executar o Seeder: Para popular a tabela produtos com os dados gerados pelo seeder:

```
php artisan db:seed
```

Projeto Prático e Boas Práticas

Projeto Prático: Desenvolvimento de uma aplicação CRUD simples - Produto

Vamos Desenvolver um Projeto Prático com uma aplicação de CRUD Simples.

Vamos criar um projeto chamado GerenciamentoProduto.

Instalar Laravel

execute o comando:

```
composer create-project --prefer-dist laravel/laravel GerenciamentoProduto
```

Configurar o Banco de Dados

Abra o arquivo .env e configure as informações do banco de dados:

```
DB_CONNECTION=postgres
```

```
DB_HOST=127.0.0.1
```

```
DB_PORT=5432
```

```
DB_DATABASE=gerenciamento_produto
```

DB_USERNAME=postgres
DB_PASSWORD=postgres

Criar o Modelo do Produto

Crie o modelo Product:

```
php artisan make:model Produto
```

Criar a Migração do Produto

Crie a migração para a tabela produtos:

```
php artisan make:migration create_produtos_table
```

No arquivo de migração gerado em
database/migrations/xxxx_xx_xx_xxxxxx_create_produtos_table.php, defina a estrutura da
tabela:

```
public function up()
{
    Schema::create('products', function (Blueprint $table) {
        $table->id();
        $table->string('nome');
        $table->text('descricao')->nullable();
        $table->decimal('preco', 8, 2);
        $table->timestamps();
    });
}
```

Executar as Migrações

Execute o comando para criar a tabela no banco de dados:

```
php artisan migrate
```

Criar o Controller do Produto

Crie o controlador do produto com recursos de CRUD:

```
php artisan make:controller ProdutoController --resource
```

Definir Rotas

No arquivo routes/web.php, adicione as rotas para o controlador de produtos:

```
use App\Http\Controllers\ProdutoController;
```

```
Route::resource('produtos', ProdutoController::class);
```

Implementar o Controller

No arquivo app/Http/Controllers/ProdutoController.php, implemente os métodos para CRUD:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Produto;

class ProdutoController extends Controller
{

    public function index()
```

```
{
    $produtos = Produto::all();
    return view('produtos', compact('produtos'));
}

public function create()
{
    return view('produtos.create');
}

public function store(Request $request)
{
    $request->validate([
        'nome' => 'required',
        'preco' => 'required|decimal',
    ]);

    Produto::create($request->all());

    return redirect()->route('produtos.index')
        ->with('success', 'Produto criado com sucesso.');
```

```
}

public function show(Produto $produto)
{
    return view('produtos.show', compact('produto'));
}

public function edit(Produto $produto)
{
    return view('produto.edit', compact('produto'));
}

public function update(Request $request, Produto $produto)
{
    $request->validate([
        'nome' => 'required',
        'preco' => 'required|decimal',
    ]);
    $produto->update($request->all());
}
```



```

        return redirect()->route('produtos.index')
        ->with('success', 'Produto atualizado com sucesso.');
```

}

```

public function destroy(Produto $produto)
{
    $produto->delete();

    return redirect()->route('produtos.index')
    ->with('success', 'Produto Deletado com Sucesso.');
```

}

}

Criar as Views

Crie as views necessárias em resources/views/produtos:

index.blade.php (Listar produtos)

create.blade.php (Formulário de criação)

edit.blade.php (Formulário de edição)

show.blade.php (Mostrar detalhes do produto)

Views Para Layout:

Crie as views necessárias para layout em resources/views/layout:

master.blade.php(inclusão de CSS, JS)

master.blade.php:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Gerenciamento de Produtos</title>
```

```
    <link rel="stylesheet"
```

```
href="https://cdnjs.cloudflare.com/ajax/libs/twitter-bootstrap/4.5.2/css/bootstrap.min.css">
```

```
</head>
```

</html>

index.blade.php:

```
<div class="container">
```

```
  <h1 class="my-4">Produtos</h1>
```

```
  @if ($message = Session::get('sucesso'))
```

```
    <div class="alert alert-success">
```

```
      <p>{{ $message }}</p>
```

```
    </div>
```

```
  @endif
```

```
  <a class="btn btn-success mb-2" href="{{ route('produtos.create') }}"> Criar Novo Produto</a>
```

```
  <table class="table table-bordered">
```

```
    <tr>
```

```
      <th>No</th>
```

```
      <th>Nome</th>
```

```
      <th>Descrição</th>
```

```
      <th>Preço</th>
```

```
      <th width="280px">Ação</th>
```

```
    </tr>
```

```
    @foreach ($produtos as $produto)
```

```
      <tr>
```

```
        <td>{{ $loop->iteration }}</td>
```

```
        <td>{{ $produto->nome }}</td>
```

```
        <td>{{ $produto->descricao }}</td>
```

```
        <td>{{ $produto->preco }}</td>
```

```
        <td>
```

```
          <form action="{{ route('produtos.destroy', $produto->id) }}" method="POST">
```

```
            <a class="btn btn-info" href="{{ route('produtos.show', $produto->id)
```

```
          }}">Mostrar</a>
```

```
<a class="btn btn-primary" href="{{ route('produtos.edit', $produto->id)
}}">Editar</a>
```

```
        @csrf
        @method('DELETE')
        <button type="submit" class="btn btn-danger">Deletar</button>
    </form>
</td>
</tr>
@endforeach
</table>
</div>
```

create.blade.php:

```
<div class="container">
    <h1 class="my-4">Criar Produto</h1>

    @if ($errors->any())
        <div class="alert alert-danger">
            <strong>Oops!</strong> Houve alguns problemas com sua entrada.<br><br>
            <ul>
                @foreach ($errors->all() as $error)
                    <li>{{ $error }}</li>
                @endforeach
            </ul>
        </div>
    @endif

    <form action="{{ route('produtos.store') }}" method="POST">
        @csrf

        <div class="form-group">
            <label for="nome">Nome:</label>
            <input type="text" name="nome" class="form-control" placeholder="Nome">
        </div>
    </form>
</div>
```

</div>

<div class="form-group">

<label for="descricao">Descrição:</label>

<textarea name="descricao" class="form-control" placeholder="Descrição"></textarea>

</div>

<div class="form-group">

<label for="preco">Preço:</label>

<input type="text" name="preco" class="form-control" placeholder="Preço">

</div>

<button type="submit" class="btn btn-primary">Enviar</button>

</form>

</div>

edit.blade.php:

<div class="container">

<h1 class="my-4">Editar Produto</h1>

@if (\$errors->any())

<div class="alert alert-danger">

Oops! Houve alguns problemas com sua entrada.

@foreach (\$errors->all() as \$error)

{{ \$error }}

@endforeach

</div>

@endif

<form action="{{ route('produtos.update', \$produto->id) }}" method="POST">

@csrf

```
@method('PUT')
```

```
<div class="form-group">  
  <label for="nome">Nome:</label>  
  <input type="text" name="nome" class="form-control" value="{{ $produto->nome }}">  
</div>
```

```
<div class="form-group">  
  <label for="descricao">Descrição:</label>  
  <textarea name="descricao" class="form-control">{{ $produto->descricao }}</textarea>  
</div>
```

```
<div class="form-group">  
  <label for="preco">Preço:</label>  
  <input type="text" name="preco" class="form-control" value="{{ $produto->preco }}">  
</div>
```

```
<button type="submit" class="btn btn-primary">Enviar</button>
```

```
</form>
```

```
</div>
```

show.blade.php:

```
<div class="container">  
  <h1 class="my-4">Detalhes do Produto</h1>
```

```
<div class="form-group">  
  <strong>Nome:</strong>  
  {{ $produto->nome }}  
</div>
```

```
<div class="form-group">  
  <strong>Descrição:</strong>  
  {{ $produto->descricao }}
```

```
</div>
```

```
<div class="form-group">  
    <strong>Preço:</strong>  
    {{ $produto->preco }}  
</div>
```

```
<a class="btn btn-primary" href="{{ route('produtos.index') }}">Voltar</a>  
</div>
```

Testar o CRUD

Inicie o servidor de desenvolvimento:

```
php artisan serve
```

Projeto Prático: Desenvolvimento de uma aplicação CRUD simples - Vaga

Vamos Desenvolver um Projeto Prático com uma aplicação de CRUD Simples.
Vamos criar um projeto chamado CrudVagas.

Instalar Laravel

execute o comando:

```
composer create-project --prefer-dist laravel/laravel CrudVagas
```

Configurar o Banco de Dados

Abra o arquivo .env e configure as informações do banco de dados:

```
DB_CONNECTION=postgres
```

```
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=crud_vagas
DB_USERNAME=postgres
DB_PASSWORD=postgres
```

Criar o Modelo de Vaga

Crie o modelo Product:

```
php artisan make:model Vaga -m
```

Modifique as Migração de Vagas

No arquivo de migração gerado em database/migrations/xxxx_xx_xx_XXXXXX_create_vagas_table.php, defina a estrutura da tabela:

```
public function up()
{
    Schema::create('products', function (Blueprint $table) {
        $table->id();
        $table->string('titulo');
        $table->text('descricao');
        $table->string('setor');
        $table->decimal('remuneracao', 8, 2);
        $table->string('empresa');
        $table->timestamps();
    });
}
```

Executar as Migrações

Execute o comando para criar a tabela no banco de dados:

```
php artisan migrate
```

Criar o Controller de Vagas

Crie o controlador do produto com recursos de CRUD:

```
php artisan make:controller VagaController --resource
```

Definir Rotas

No arquivo routes/web.php, adicione as rotas para o controlador de produtos:

```
use App\Http\Controllers\VagaController;  
  
Route::resource('vagas', VagaController::class);
```

Implementar o Controller

No arquivo app/Http/Controllers/VagaController.php, implemente os métodos para CRUD:

```
<?php  
  
namespace App\Http\Controllers;  
  
use Illuminate\Http\Request;  
use App\Models\Vaga;  
  
class ProdutoController extends Controller  
{  
  
    public function index()  
    {  
    }  
}
```



```
{
    $vagas = Vaga::all();
    return view('vagas.index', compact('vagas'));
}

public function create()
{
    return view('vagas.create');
}

public function store(Request $request)
{
    $request->validate([
        'titulo' => 'required',
        'descricao' => 'required',
        'setor' => 'required',
        'remuneracao' => 'required|numeric',
        'empresa' => 'required',
    ]);

    Vaga::create($request->all());

    return redirect()->route('vagas.index')
        ->with('success', 'Vaga criado com sucesso.');
```

```
}

public function show(Vaga $vaga)
{
    return view('vagas.show', compact('vaga'));
}

public function edit(Vaga $vaga)
{
    return view('vaga.edit', compact('vaga'));
}

public function update(Request $request, Vaga $vaga)
{
    $request->validate([
        'titulo' => 'required',
```

```

        'descricao' => 'required',
        'setor' => 'required',
        'remuneracao' => 'required|numeric',
        'empresa' => 'required',

    ]);
    $vaga->update($request->all());
    return redirect()->route('vagas.index')
        ->with('success', 'Vaga atualizado com sucesso.');
```

```

    }

    public function destroy(Vaga $vaga)
    {
        $vaga->delete();

        return redirect()->route('vagas.index')
            ->with('success', 'Vaga Deletado com Sucesso.');
```

```

    }
}
```

Criar as Views

Crie as views necessárias em resources/views/vagas:

index.blade.php (Listar produtos)

create.blade.php (Formulário de criação)

edit.blade.php (Formulário de edição)

show.blade.php (Mostrar detalhes do produto)

Views Para Layout:

Crie as views necessárias para layout em resources/views/layout:

master.blade.php(inclusão de CSS, JS)

master.blade.php:

```
<!DOCTYPE html>
```

```

<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/twitter-bootstrap/4.5.2/css/b
ootstrap.min.css">
</head>
<body>

</body>
</html>

```

index.blade.php:

```

@extends('layouts.master')

@section('content')
<div class="container">
    <h1 class="my-4">Vagas</h1>

    @if ($message = Session::get('success'))
        <div class="alert alert-success">
            <p>{{ $message }}</p>
        </div>
    @endif

    <a class="btn btn-success mb-2" href="{{ route('vagas.create') }}">
Criar Nova Vaga</a>

    <table class="table table-bordered">
        <tr>
            <th>N°</th>
            <th>Título</th>
            <th>Descrição</th>
            <th>Setor</th>
            <th>Remuneração</th>

```

```

        <th>Empresa</th>
        <th width="280px">Ação</th>
    </tr>
    @foreach ($vagas as $vaga)
    <tr>
        <td>{{ $loop->iteration }}</td>
        <td>{{ $vaga->titulo }}</td>
        <td>{{ $vaga->descricao }}</td>
        <td>{{ $vaga->setor }}</td>
        <td>{{ $vaga->remuneracao }}</td>
        <td>{{ $vaga->empresa }}</td>
        <td>
            <form action="{{ route('vagas.destroy', $vaga->id) }}"
method="POST">
                <a class="btn btn-info" href="{{ route('vagas.show',
$vaga->id) }}">Mostrar</a>
                <a class="btn btn-primary" href="{{
route('vagas.edit', $vaga->id) }}">Editar</a>

                @csrf
                @method('DELETE')
                <button type="submit" class="btn
btn-danger">Deletar</button>
            </form>
        </td>
    </tr>
    @endforeach
</table>
</div>

```

create.blade.php:

```

@extends('layouts.master')

@section('content')

<div class="container">
    <h1 class="my-4">Criar Vaga</h1>

```

```
@if ($errors->any())
    <div class="alert alert-danger">
        <strong>Oops!</strong> Houve alguns problemas com sua
entrada.<br><br>
        <ul>
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    </div>
@endif

<form action="{{ route('vagas.store') }}" method="POST">
    @csrf

    <div class="form-group">
        <label for="nome">Título:</label>
        <input type="text" name="titulo" class="form-control"
placeholder="Título">
    </div>

    <div class="form-group">
        <label for="setor">Setor:</label>
        <input type="text" name="setor" class="form-control"
placeholder="Setor">
    </div>

    <div class="form-group">
        <label for="descricao">Descrição:</label>
        <textarea name="descricao" class="form-control"
placeholder="Descrição"></textarea>
    </div>

    <div class="form-group">
        <label for="remuneracao">Remuneração:</label>
        <input type="text" name="remuneracao" class="form-control"
placeholder="Remuneração">
    </div>
```

```

        <div class="form-group">
            <label for="empresa">Empresa:</label>
            <input type="text" name="empresa" class="form-control"
placeholder="Empresa">
        </div>

        <button type="submit" class="btn btn-primary">Enviar</button>
    </form>
</div>

```

edit.blade.php:

```

<div class="container">
    <h1 class="my-4">Editar Produto</h1>

    @if ($errors->any())
        <div class="alert alert-danger">
            <strong>Oops!</strong> Houve alguns problemas com sua entrada.<br><br>
            <ul>
                @foreach ($errors->all() as $error)
                    <li>{{ $error }}</li>
                @endforeach
            </ul>
        </div>
    @endif

    <form action="{{ route('produtos.update', $produto->id) }}" method="POST">
        @csrf
        @method('PUT')

        <div class="form-group">
            <label for="nome">Nome:</label>
            <input type="text" name="nome" class="form-control" value="{{ $produto->nome }}">
        </div>
    </form>

```

```
<div class="form-group">
    <label for="descricao">Descrição:</label>
    <textarea name="descricao" class="form-control">{{ $produto->descricao }}</textarea>
</div>

<div class="form-group">
    <label for="preco">Preço:</label>
    <input type="text" name="preco" class="form-control" value="{{ $produto->preco }}">
</div>

<button type="submit" class="btn btn-primary">Enviar</button>
</form>
</div>
```

show.blade.php:

```
@extends('layouts.master')

@section('content')
<div class="container">
    <h1 class="my-4">Detalhes da Vaga</h1>

    <div class="form-group">
        <strong>Título:</strong>
        {{ $vaga->titulo }}
    </div>

    <div class="form-group">
        <strong>Setor:</strong>
        {{ $vaga->setor }}
    </div>

    <div class="form-group">
        <strong>Descrição:</strong>
        {{ $vaga->descricao }}
    </div>
</div>
```

```
<div class="form-group">
    <strong>Remuneração:</strong>
    {{ $vaga->remuneracao }}
</div>

<div class="form-group">
    <strong>Empresa:</strong>
    {{ $vaga->empresa }}
</div>

<a class="btn btn-primary" href="{{ route('vagas.index')
}}">Voltar</a>
</div>
```

Testar o CRUD

Inicie o servidor de desenvolvimento:

```
php artisan serve
```

Projeto Portal de Empregos - Avaliação Formativa Tarde

Projeto:

Sistema Web de Gerenciamento de Empregos e Currículo

Descrição do Projeto:

Site de vagas de emprego onde os usuários possam cadastrar seus currículos e se candidatar a vagas, enquanto as empresas possam cadastrar suas informações e publicar oportunidades de emprego.

Elaboração do Escopo e Objetivos do Projeto:

Escopo:

O projeto consiste no desenvolvimento de uma plataforma web de vagas de emprego utilizando Laravel. O sistema permitirá que candidatos criem contas, cadastrem seus currículos e se candidatem a vagas de emprego. Empresas poderão se cadastrar, criar e gerenciar vagas de emprego, bem como visualizar as candidaturas recebidas. O sistema incluirá também funcionalidades de pesquisa e filtragem de vagas, autenticação e autorização de usuários, e dashboards para administração das funcionalidades.

Objetivos:

Específicos:

- Desenvolver um sistema de cadastro e autenticação para candidatos e empresas.
- Implementar funcionalidades para candidatos visualizarem, filtrarem e se candidatarem a vagas de emprego.
- Permitir que empresas registrem e gerenciem suas vagas de emprego.
- Criar dashboards específicos para candidatos e empresas, com funcionalidades relevantes para cada perfil de usuário.
- Garantir a segurança dos dados dos usuários e a integridade das transações dentro do sistema.

Mensuráveis:

- Implementação de uma plataforma web funcional, com um mínimo de 80% de cobertura de testes unitários.
- Redução do tempo de resposta do sistema para consultas de vagas para menos de 2 segundos.
- Atingir um nível de satisfação de usuários acima de 85% em testes de usabilidade.
- Garantir que o sistema suporte até 1000 usuários simultâneos sem degradação de desempenho.

Atingíveis:

- Utilizar frameworks e bibliotecas bem documentadas (como Laravel) para assegurar o cumprimento dos prazos e qualidade do código.

- Definir entregas em ciclos de desenvolvimento ágil (sprints), com revisões e feedback constantes.
- Estabelecer uma equipe de desenvolvimento com experiência em Laravel e projetos similares.

Relevantes:

- Atender à necessidade crescente de plataformas eficientes e seguras para busca e candidatura de vagas de emprego.
- Facilitar a comunicação entre candidatos e empregadores, otimizando o processo de recrutamento.
- Prover uma solução escalável que possa ser expandida conforme o crescimento da base de usuários.

Temporais:

- Concluir o desenvolvimento do sistema em 6 meses, com entregas iterativas a cada 4 semanas.
- Implementar e testar o módulo de autenticação até o final do segundo mês.
- Finalizar o módulo de candidaturas até o quarto mês.
- Realizar testes beta e ajustes finais no último mês do cronograma.

Planejamento do Projeto:**Cronograma:**

1:

- Configuração do ambiente de desenvolvimento.
- Implementação do sistema de autenticação e autorização.
- Criação das páginas de registro e login para candidatos e empresas.

2:

- Desenvolvimento do módulo de cadastro e gestão de vagas para empresas.
- Implementação da funcionalidade de pesquisa e filtragem de vagas.

- Criação do módulo de candidaturas para candidatos.

3:

- Implementação dos dashboards para candidatos e empresas.
- Testes de usabilidade e performance.
- Ajustes e melhorias com base no feedback.

4:

- Testes beta com um grupo de usuários selecionados.
- Correção de bugs e refinamento da interface.
- Preparação para lançamento e implementação de estratégias de marketing.

Recursos:

Equipe:

- 1 Gerente de Projetos
- 2 Desenvolvedores Laravel
- 1 Designer UI/UX
- 1 Especialista em Testes e QA
- 1 Analista de Marketing

Tecnologias:

- Laravel (Framework PHP)
- MySQL (Banco de Dados)
- Bootstrap (Front-end)
- AWS/Heroku (Hospedagem)
- Git/GitHub (Controle de Versão)

Ferramentas de Gestão:

- Jira/Trello para gerenciamento de tarefas

- Teams para comunicação interna
- Figma para design de interfaces

Análise de Riscos:

Risco: Atrasos no cronograma devido a requisitos não previstos.

- Mitigação: Realizar reuniões regulares com stakeholders para garantir o alinhamento das expectativas e ajustes de escopo.

Risco: Falhas de segurança que comprometem dados dos usuários.

- Mitigação: Implementar práticas de segurança desde o início do desenvolvimento e realizar auditorias de segurança regularmente.

Risco: Baixa adesão de usuários no lançamento.

- Mitigação: Desenvolver uma estratégia de marketing eficaz, incluindo campanhas de pré-lançamento para gerar interesse.

Risco: Sobrecarga no sistema com o aumento rápido de usuários.

- Mitigação: Implementar soluções escaláveis e realizar testes de carga para garantir a performance do sistema sob alta demanda.

Desenvolvimento:

Diagramas de Desenvolvimento

- Diagrama de Classe
- Diagrama de Uso
- Diagrama de Fluxo

Criação do Projeto

composer create-project laravel/laravel PortalVagas --prefer-dist

Criar o Banco de Dados no PostgreSQL

```
CREATE DATABASE portal_vagas;
```

Preparar o .env

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1 #localhost
DB_PORT=5432
DB_DATABASE=portal_vagas
DB_USERNAME=postgres
DB_PASSWORD=postgres
```

Rodar as Migrações do Banco de Dados(dentro do projeto)

```
php artisan migrate
```

Criar os Modelos e as Migrations(modelos de tabelas do BD)

```
php artisan make:model Usuario -m
php artisan make:model Empresa -m
php artisan make:model Vaga -m
php artisan make:model Inscricao -m
```

Configuração dos Modelos

Configure os modelos para refletirem os relacionamentos definidos nas migrações.

Modelo Usuario

Edite o modelo Usuario em app/Models/Usuario.php:

```
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class Usuario extends Authenticatable
{
    use Notifiable, HasFactory;
    protected $fillable = [
        'nome', 'email', 'password',
    ];
}
```

```

];
protected $hidden = [
    'password', 'remember_token',
];
public function inscricoes()
{
    return $this->hasMany(Inscricao::class);
}
}

```

Modelo Empresa

Edite o modelo Empresa em app/Models/Empresa.php:

```

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class Empresa extends Authenticatable
{
    use Notifiable, HasFactory;
    protected $fillable = [
        'nome', 'email', 'password', 'nomeEmpresa',
        'descricaoEmpresa'
    ];
    protected $hidden = [
        'password', 'remember_token',
    ];
    public function vagas()
    {
        return $this->hasMany(Vaga::class);
    }
}

```

Modelo Vaga

Edite o modelo Vaga em app/Models/Vaga.php:

```

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Vaga extends Model
{
    use HasFactory;

    protected $fillable = [
        'titulo', 'descricao', 'localizacao', 'salario',
        'empresa_id',
    ];

    public function empresa()
    {
        return $this->belongsTo(Empresa::class);
    }

    public function inscricoes()
    {
        return $this->hasMany(Inscricao::class);
    }
}

```

Modelo Inscricao

Edite o modelo Inscricao em app/Models/Inscricao.php:

```

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Inscricao extends Model
{
    use HasFactory;

    protected $fillable = [

```

```

        'usuario_id', 'vaga_id', 'status',
    ];

    public function usuario()
    {
        return $this->belongsTo(Usuario::class);
    }

    public function vaga()
    {
        return $this->belongsTo(Vaga::class);
    }
}

```

Definir as Migrações:

Edite o arquivo de migration criado para Usuario em database/migrations/{timestamp}_create_usuarios_table.php:

```

public function up(): void
{
    Schema::create('usuarios', function (Blueprint $table) {
        $table->id();
        $table->string('nome');
        $table->string('email')->unique();
        $table->timestamp('email_verified_at')->nullable();
        $table->string('password');
        $table->rememberToken();
        $table->timestamps();
    });
}

/**
 * Reverse the migrations.
 */
public function down(): void
{
    Schema::dropIfExists('usuarios');
}

```


Edite o arquivo de migration criado para Empresa em
database/migrations/{timestamp}_create_empresas_table.php:

```
public function up(): void
{
    Schema::create('empresas', function (Blueprint $table) {
        $table->id();
        $table->string('nome');
        $table->string('email')->unique();
        $table->string('nome_empresa');
        $table->text('descricao_empresa');
        $table->timestamp('email_verified_at')->nullable();
        $table->string('password');
        $table->rememberToken();
        $table->timestamps();
    });
}

public function down(): void
{
    Schema::dropIfExists('empresas');
}
```

Edite o arquivo de migration criado para Vaga em
database/migrations/{timestamp}_create_vagas_table.php:

```
public function up(): void
{
    Schema::create('vagas', function (Blueprint $table) {
        $table->id();
        $table->string('titulo');
        $table->text('descricao');
        $table->string('localizacao');
        $table->decimal('salario', 10, 2)->nullable();

        $table->foreignId('empresa_id')->constrained('empresas')->
            onDelete('cascade'); // Relaciona a vaga à empresa
    });
}
```

```

        $table->timestamps();
    });

}

public function down(): void
{
    Schema::dropIfExists('vagas');
}

```

Edite o arquivo de migration criado para Inscrição em database/migrations/{timestamp}_create_inscricoes_table.php:

```

public function up(): void
{
    Schema::create('inscricoes', function (Blueprint $table) {
        $table->id();

        $table->foreignId('usuario_id')->constrained('usuarios')->
            onDelete('cascade'); // Relaciona a inscrição ao
usuário

        $table->foreignId('vaga_id')->constrained('vagas')->
            onDelete('cascade'); // Relaciona a inscrição à vaga
        $table->string('status')->default('pendente');
        $table->timestamps();
    });
}

public function down(): void
{
    Schema::dropIfExists('inscricoes');
}

```

Rodar as Migrações

Agora que todas as migrações foram criadas, execute:

```
php artisan migrate
```

Implementar a Lógica de Autenticação no UsuarioController

Criar o UsuarioController:

```
php artisan make:controller UsuarioController
```

Adicionar Métodos de Login, Registro e Logout:

No arquivo app/Http/Controllers/UsuarioController.php, adicione os métodos para gerenciar a autenticação:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Hash;
use App\Models\Usuario;

class UsuarioController extends Controller
{
    // Exibir o formulário de login
    public function showLoginForm()
    {
        return view('usuarios.login');
    }

    // Processar o login do usuário
    public function login(Request $request)
    {
        // Validações para o login
        $credentials = $request->validate([
            'email' => ['required', 'email'],
            'password' => ['required'],
        ]);

        // Tenta autenticar com o guard 'usuario'
        if (Auth::guard('usuario')->attempt($credentials)) {
```

```

        $request->session()->regenerate(); // Regenera a sessão
para evitar fixação de sessão
        return redirect()->intended('/dashboard');
    }

    // Se falhar, retorna com erro
    return back()->withErrors([
        'email' => 'As credenciais não correspondem aos nossos
registros.',
    ])->onlyInput('email');
}

// Exibir o formulário de registro
public function showRegistroForm()
{
    return view('usuarios.registro');
}

// Processar o registro de um novo usuário
public function registro(Request $request)
{
    // Validações para o registro
    $request->validate([
        'nome' => 'required|string|max:255',
        'email' =>
'required|string|email|max:255|unique:usuarios',
        'password' => 'required|string|min:8|confirmed',
    ]);

    // Cria um novo usuário
    $usuario = Usuario::create([
        'nome' => $request->nome,
        'email' => $request->email,
        'password' => Hash::make($request->password),
    ]);

    // Faz login automático do novo usuário
    Auth::guard('usuario')->login($usuario);

    return redirect('/dashboard');
}

```

```

    }

    // Realizar o logout do usuário
    public function logout(Request $request)
    {
        Auth::guard('usuario')->logout(); // Logout do guard
        'usuario'
        $request->session()->regenerateToken(); // Regenera o token
        da sessão

        $request->session()->invalidate();
        $request->session()->regenerate(); // Invalida a sessão

        return redirect('/');
    }
}

```

Criar as Rotas para usuário Controller:

No arquivo routes/web.php, defina as rotas para login, registro e logout utilizando o UsuarioController:

```

<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\UsuarioController;

// Rota para exibir a homePage
Route::get('/', function() {return view('home');});

// Rota para exibir o formulário de login
Route::get('/login', [UsuarioController::class, 'showLoginForm'])->
name('usuarios.login');

// Rota para processar o login
Route::post('/login', [UsuarioController::class, 'login'])->
name('usuarios.login');

// Rota para exibir o formulário de registro

```

```

Route::get('/registro', [UsuarioController::class,
'showRegisterForm'])->
name('usuarios.register');

// Rota para processar o registro
Route::post('/registro', [UsuarioController::class, 'register'])->
name('usuarios.register');

// Rota para logout
Route::post('/logout', [UsuarioController::class, 'logout'])->
name('usuarios.logout');

// Rota para o dashboard, protegida por autenticação
Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware('auth')->name('dashboard');

```

Prepara as View do Projeto

Crie o Arquivo layouts.app.blade.php

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{{ config('app.name', 'Portal Vagas') }}</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.
css" rel="stylesheet">
</head>
<body>

    @include('parts.header')

    <div class="container">
        @yield('content')
    </div>

```

```
@include('parts.footer')

</body>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle
.min.js"></script>
</html>
```

Criar a home.blade.php

Criar a usuarios.registro.blade.php

```
@extends('layouts.app')

@section('content')
{{-- formulario --}}
<div class="container">
    <h1>Registrar-se</h1>
    <form method="POST" action="{{ route('usuarios.registro') }}">
        @csrf

        <div class="form-group">
            <label for="name">Nome</label>
            <input type="text" name="name" class="form-control" required>
        </div>

        <div class="form-group">
            <label for="email">Email</label>
            <input type="email" name="email" class="form-control"
required>
        </div>

        <div class="form-group">
            <label for="password">Senha</label>
            <input type="password" name="password" class="form-control"
required>
        </div>
```

```

        <div class="form-group">
            <label for="password_confirmation">Confirme a Senha</label>
            <input type="password" name="password_confirmation"
class="form-control" required>
        </div>

        <button type="submit" class="btn
btn-primary">Registrar-se</button>
    </form>
</div>

```

```
@endsection
```

Criar a Página usuarios.login.blade.php

```

@extends('layouts.app')

@section('content')
<div class="container">
    <h1>Login</h1>
    <form method="POST" action="{{ route('usuarios.login') }}">
        @csrf

        <div class="form-group">
            <label for="email">Email</label>
            <input type="email" name="email" class="form-control" required
autofocus>
        </div>

        <div class="form-group">
            <label for="password">Senha</label>
            <input type="password" name="password" class="form-control"
required>
        </div>

        <button type="submit" class="btn btn-primary">Login</button>
    </form>
</div>
@endsection

```


<https://pt.stackoverflow.com/questions/279238/usu%C3%A1rio-n%C3%A3o-fica-autenticado-auth>

auth.php editar para:

```
<?php

return [

    /*
    |-----
    |
    | Authentication Defaults
    |-----
    |
    | This option defines the default authentication "guard" and password
    | reset "broker" for your application. You may change these values
    | as required, but they're a perfect start for most applications.
    |
    */

    'defaults' => [
        'guard' => 'usuario',
        'passwords' => 'users',
    ],

    /*
    |-----
    |
    | Authentication Guards
    |-----
    |
    |
```

```
|
| Next, you may define every authentication guard for your
application.
| Of course, a great default configuration has been defined for you
| which utilizes session storage plus the Eloquent user provider.
|
| All authentication guards have a user provider, which defines how
the
| users are actually retrieved out of your database or other storage
| system used by the application. Typically, Eloquent is utilized.
|
| Supported: "session"
|
*/
```

```
'guards' => [
    'usuario' => [
        'driver' => 'session',
        'provider' => 'usuario',
    ],
    'api' => [
        'driver' => 'token',
        'provider' => 'users',
    ],
],
/*
```

```
|-----
-

```

| User Providers

```
|-----
-

```

```
|
| All authentication guards have a user provider, which defines how
the
| users are actually retrieved out of your database or other storage
| system used by the application. Typically, Eloquent is utilized.
|
```

```

    | If you have multiple user tables or models you may configure
multiple
    | providers to represent the model / table. These providers may then
    | be assigned to any extra authentication guards you have defined.
    |
    | Supported: "database", "eloquent"
    |
    */

    'providers' => [
        'usuario' => [
            'driver' => 'eloquent',
            'model' => App\Models\Usuario::class,
        ],

        // 'users' => [
        //     'driver' => 'database',
        //     'table' => 'users',
        // ],
    ],

    /*

|-----
-
    | Resetting Passwords

|-----
-
    |
    | These configuration options specify the behavior of Laravel's
password
    | reset functionality, including the table utilized for token storage
    | and the user provider that is invoked to actually retrieve users.
    |
    | The expiry time is the number of minutes that each reset token will
be
    | considered valid. This security feature keeps tokens short-lived so
    | they have less time to be guessed. You may change this as needed.
    |

```

```

        | The throttle setting is the number of seconds a user must wait
before
        | generating more password reset tokens. This prevents the user from
        | quickly generating a very large amount of password reset tokens.
        |
        */

        'passwords' => [
            'users' => [
                'provider' => 'users',
                'table' => 'password_resets',
                'expire' => 60,
            ],
        ],

        /*

|-----|
-
        | Password Confirmation Timeout

|-----|
-
        |
        | Here you may define the amount of seconds before a password
confirmation
        | window expires and users are asked to re-enter their password via
the
        | confirmation screen. By default, the timeout lasts for three hours.
        |
        */

        'password_timeout' => env('AUTH_PASSWORD_TIMEOUT', 10800),

];

```

Criar uma NavBar de navegação Modificada conforme a etapa do usuário

views/parts/header.blade.php

```
@if(Auth::check())
    @if (
        request() //captura os dados da requisição
        ->user() //captura o usuario que fez a requisição
        ->where('tipo','empresa') // a partir de model binding ele
        procura se o tipo é empresa
        ->first() //transforma o resultado da consulta em um
        valor boolean, retornando true or false
    )
        <div>
            Olá empresa
        </div>
    @else
        <div>
            Olá Usuário
        </div>
    @endif
@endif
```

Projeto Loja de Ferramentas - Avaliação Formativa Noite

Projeto:

Descrição do Projeto:

Elaboração do Escopo e Objetivos do Projeto:

Planejamento do Projeto:

Análise de Riscos:

Desenvolvimento:

Diagrama de Desenvolvimento:

Classes Principais:

- Usuário -
Atributos
 - String - Nome
 - String - Email
 - String - Senha
 - Enum - Eh_AdmMétodos
 - Create -
 - Auth -
 - Update -
- Produto -
Atributos
 - Nome
 - Descrição
 - Preço
 - Quantidade
 - CategoriaMétodos
 - CRUD
- Carrinho -
Atributos

Id_Usuario
Id_Produto
Quantidade Produto
Valor_Total

Métodos:

Adicionar
Remover
Finalizar

- Pagamento

Atributos

Id_Usuário
Id_Carrinho
Valor
Data
Status

Relacionamentos:

Usuario => Produtos

Usuário => Carrinho

Carrinho => Pagamento

Diagrama de Casos de Uso

Usuários

Cadastro
Login
Comprar
CRUD Produtos(ADM)

Produtos

Adicionar e Remover Carrinho

Carrinho

Status Pagamento

Diagrama de Fluxo

Página Inicial(Home)

|

Login -> Fazer Cadastro -> Fazer Login

|

Verificar Tipo de Usuários

|

Cliente -> Ver Produtos -> Carrinho -> Finalizar -> Pagamento

Adm -> CRUD PRodutos -> Dashboard de Vendas

|

Logout -> Home

Passo a Passo do Desenvolvimento

Criar Projeto Laravel

```
composer create-project --prefer-dist laravel/laravel LojaFerramentas
```

Configuração do Banco de Dados

No arquivo .env, configure a conexão com o banco de dados:

```
DB_CONNECTION=pgsql  
DB_HOST=127.0.0.1  
DB_PORT=5432  
DB_DATABASE=loja_ferramentas  
DB_USERNAME=postgres  
DB_PASSWORD=postgres
```

Execute a migração

```
php artisan migrate
```

Modelos e Migrações

Modificar o User Model

```
protected $fillable = [
```



```

        'name',
        'email',
        'password',
        'endereco',
        'telefone',
        'tipo_usuario'
    ];

```

// Adicionar no Final do Model

```

// Verifica se o usuário é um administrador
public function isAdmin()
{
    return $this->tipo_usuario === 'administrador';
}

// Verifica se o usuário é um cliente
public function isClient()
{
    return $this->tipo_usuario === 'cliente';
}

```

Modificar o Migration - User

Adicionar alguns atributos para armazenar informações adicionais. em
database/migrations/xxxx_xx_xx_create_users_table.php

```

Schema::table('users', function (Blueprint $table) {
    $table->string('endereco')->nullable();
    $table->string('telefone')->nullable();
    $table->enum('tipo_usuario', ['cliente', 'administrador'])->default('cliente');
}

```

Criar e Modificar os Models e Migrations Para Produtos e Carrinho

```

php artisan make:model Produto -m
php artisan make:model Carrinho -m
php artisan make:model Pagamento -m

```

Criar o Controller Para User

```

php artisan make:controller UserController

```

Editar o Arquivo App/Http/Controller/UserController.php

Criar os Mecanismos de Autenticação

Criar as View

usuarios.registro.blade.php

```
{{-- formularios de cadastro --}}
@extends('layouts.app')

@section('content')
    {{-- formulario --}}
    <div class="container">
        <h1>Registrar-se</h1>
        <form method="POST" action="{{ route('usuarios.registro') }}">
            @csrf

            <div class="form-group">
                <label for="name">Nome</label>
                <input type="text" name="name" class="form-control" required>
            </div>

            <div class="form-group">
                <label for="email">Email</label>
                <input type="email" name="email" class="form-control
                @error('email')is-invalid @enderror" value="{{old('email')}}" required>
                @error('email')
                <div class="invalid-feedback">{{ $message }}</div>
                @enderror
            </div>

            <div class="form-group">
                <label for="password">Senha</label>
```

```

        <input type="password" name="password" class="form-control
@error('password')is-invalid @enderror" required>
        @error('password')
        <div class="invalid-feedback">{{$message}}</div>
        @enderror
    </div>

    <div class="form-group">
        <label for="password_confirmation">Confirme a Senha</label>
        <input type="password" name="password_confirmation"
class="form-control" required>
    </div>

    <button type="submit" class="btn
btn-primary">Registrar-se</button>
</form>
</div>

@endsection

```

Criar o views/usuarios/login.blade.php

```

@extends('layouts.app')

@section('content')
<div class="container">
    <h1>Login</h1>
    <form method="POST" action="{{ route('usuarios.login') }}">
        @csrf

        <div class="form-group">

```

```
<label for="email">Email</label>
<input type="email" name="email" class="form-control" required
autofocus>
</div>

<div class="form-group">
  <label for="password">Senha</label>
  <input type="password" name="password" class="form-control"
required>
</div>

<button type="submit" class="btn btn-primary">Login</button>
</form>
</div>
@endsection
```

Boas Práticas no Desenvolvimento com Laravel

- Organização do código
- Uso de middlewares
- Segurança e validação de dados

Deploy de uma Aplicação Laravel

- Preparação para o deploy
- Configuração de ambiente de produção
- Deploy em servidores compartilhados e serviços de hospedagem

Bibliografia Recomendada:

- Documentação Oficial do Laravel (<https://laravel.com/docs>)
- "Laravel: Up & Running" por Matt Stauffer

- Tutoriais e cursos online (Laracasts, YouTube)